

# Arquitectura de software según el tipo de aplicación

E.J Laferre Guevara  
7690222644 Universidad Mariano Gálvez  
Seminario de Tecnologías de la Información  
[elaferre1@miumg.edu.gt](mailto:elaferre1@miumg.edu.gt)

## Resumen

Este ensayo analiza cómo la elección de la arquitectura de software depende directamente del tipo de aplicación que se está desarrollando. Se parte de una clasificación funcional del software en seis categorías principales: software de sistema, software de aplicación, software de programación, middleware, controladores de dispositivo y software embebido. Para cada una se describe su función principal, sus características y necesidades técnicas, y las arquitecturas más adecuadas según el contexto, incluyendo patrones como la arquitectura monolítica, el microkernel, las capas, MVC y los sistemas basados en eventos.

El análisis muestra que no existe una arquitectura universal, sino que cada tipo de software exige un enfoque distinto según su propósito, su público (que puede ser una persona, otro sistema o un componente electrónico) y factores del contexto actual, como el avance de la inteligencia artificial y la ciberseguridad. Se concluye que diseñar software no se limita a resolver un problema de forma eficaz, sino que implica pensar en la escalabilidad, el mantenimiento y la legibilidad del código para quienes lo desarrollan. Este trabajo busca ofrecer una guía práctica para orientar la selección de arquitectura según las necesidades específicas de cada tipo de aplicación.

**Palabras clave:** arquitectura de software, tipos de software, diseño de sistemas, escalabilidad, patrones arquitectónicos

## Arquitectura de software según el tipo de aplicación

Toda aplicación de software nace con un propósito específico, y ese propósito debería guiar las decisiones técnicas detrás de su construcción. El software puede clasificarse de muchas formas: por función, licenciamiento, industria o caso de uso, lo que hace casi imposible reducirlo a una sola lista definitiva (Fazio, 2025).

De estos criterios, el más relevante para este análisis es la clasificación por función, es decir, según lo que el software hace. Bajo este enfoque, desde el sistema operativo de un dispositivo hasta las herramientas que sostienen la operación de una gran empresa, cada tipo de software cumple un rol distinto y enfrenta exigencias técnicas distintas.

Esta distinción es clave. Entender en qué categoría funcional se ubica una aplicación es el primer paso para decidir qué arquitectura conviene adoptar. La arquitectura no es un molde único aplicable a cualquier proyecto, pues depende del tipo de aplicación, sus requisitos de escalabilidad, su público y el contexto en el que operará. Por ello, antes de elegir una arquitectura, es indispensable partir por entender cómo se clasifica el software.

## Tipos de aplicaciones

### Software de sistema

El software de sistema es aquel que gestiona el hardware de un dispositivo y provee la plataforma base sobre la cual se ejecutan otros programas. Este tipo de software administra recursos como la memoria, el procesador y los dispositivos periféricos, funcionando en segundo plano para crear

una interfaz entre el hardware y el usuario (Finoit, 2025).

Requiere alta eficiencia, bajo consumo de recursos y estabilidad constante, ya que todo el sistema depende de su correcto funcionamiento.

Las arquitecturas más adecuadas son la monolítica de kernel, útil cuando se prioriza el rendimiento; el microkernel, preferible cuando se busca mayor estabilidad y seguridad al aislar componentes; y la arquitectura en capas, conveniente cuando se necesita organizar el sistema en niveles claros de responsabilidad.

Ejemplos de software de sistema son los sistemas operativos como Windows, macOS y Linux, así como el firmware y los controladores de dispositivos (Finoit, 2025).

### **Software de Aplicación**

El software de aplicación es aquel que realiza tareas específicas para el usuario final, como comunicación, gestión de información o procesamiento de documentos. Este tipo de software puede clasificarse en software genérico, como MS Word, que sirve para múltiples tareas, y software de propósito específico, como las aplicaciones de salud, diseñado para una función particular (Finoit, 2025).

Este software prioriza la usabilidad y la interacción con el usuario, por lo que sus necesidades giran en torno a una interfaz clara y una experiencia fluida.

Las arquitecturas más adecuadas suelen ser MVC (Modelo-Vista-Controlador) y la arquitectura en capas. El patrón MVC se usa principalmente cuando la interfaz, los datos y la lógica de negocio necesitan desarrollarse de forma separada, siendo frecuente en aplicaciones web y móviles (Dharmadasa, 2025).

Ejemplos de software de aplicación son navegadores como Chrome y Firefox, software de comunicación como Zoom y Slack, y hojas de cálculo como Excel (Finoit, 2025).

### **Software de Programación**

El software de programación proporciona a los desarrolladores las herramientas necesarias para crear otros programas. Incluye elementos como depuradores, compiladores, editores de código y entornos de desarrollo integrado (IDE), que permiten escribir, probar y gestionar código de forma eficiente (Finoit, 2025). Este tipo de software necesita ofrecer buena capacidad de depuración, integración con distintos lenguajes y herramientas de control de versiones, ya que su propósito es facilitar el trabajo técnico del equipo de desarrollo. Las arquitecturas más adecuadas suelen ser la arquitectura en capas y la arquitectura modular (plugin-based), ya que permiten separar componentes como el editor, el compilador y las extensiones. La arquitectura en capas conviene cuando se busca una organización clara entre la interfaz del usuario y el motor de procesamiento del código, mientras que la arquitectura modular es preferible cuando se necesita flexibilidad para añadir funciones o soporte de lenguajes sin afectar el núcleo del sistema. Ejemplos de software de programación son Visual Studio, Sublime Text y Eclipse (Finoit, 2025).

### **Middleware**

El middleware es un software que actúa como intermediario entre diferentes aplicaciones, sistemas o componentes, facilitando la comunicación entre ellos. Funciona como una especie de pegamento que conecta el software de aplicación con el sistema operativo, simplificando el desarrollo de soluciones distribuidas (Finoit, 2025).

Este tipo de software necesita alta confiabilidad, baja latencia y capacidad de gestionar comuni-

cación entre componentes que pueden estar en distintos servidores o plataformas.

Las arquitecturas más adecuadas suelen ser la arquitectura orientada a servicios (SOA) y la arquitectura basada en eventos. La arquitectura SOA conviene cuando se necesita que distintos sistemas se comuniquen a través de servicios bien definidos, mientras que la arquitectura basada en eventos es preferible cuando la comunicación es asíncrona, como ocurre en el middleware orientado a mensajes.

Ejemplos de middleware son el middleware de transacciones, el middleware de bases de datos y el middleware orientado a mensajes (Finoit, 2025).

### **Controladores de Dispositivo**

Los controladores de dispositivo son programas que controlan periféricos específicos conectados a la computadora, sirviendo de facilitadores para que el hardware se comunique con el sistema operativo. Cuando se conecta un dispositivo, el controlador recibe las órdenes del sistema operativo y ayuda al hardware a entenderlas, asegurando que ejecute su tarea correctamente (Finoit, 2025).

Este tipo de software necesita bajo nivel de abstracción, acceso directo al hardware y alta estabilidad, ya que errores en un controlador pueden afectar a todo el sistema.

Las arquitecturas más adecuadas suelen ser la arquitectura monolítica de kernel, cuando el controlador requiere acceso rápido y directo al hardware, y la arquitectura de microkernel, preferible cuando se busca aislar el controlador del núcleo del sistema para evitar que una falla comprometa la estabilidad general.

Ejemplos de controladores de dispositivo son los drivers de gráficos, los drivers de impresora y los drivers de dispositivos USB (Finoit, 2025).

### **Software Embebido**

El software embebido es aquel que controla el funcionamiento de un sistema embebido, diseñado para cumplir una función específica dentro de un sistema electrónico o mecánico más amplio.

Este tipo de software combina procesador, memoria y periféricos, y se encuentra comúnmente en electrodomésticos, automóviles, relojes digitales y equipos médicos (Finoit, 2025).

Este software necesita alta eficiencia, bajo consumo energético y respuesta en tiempo real, ya que suele operar con recursos de hardware limitados y en contextos donde la fiabilidad es crítica.

Las arquitecturas más adecuadas suelen ser la arquitectura basada en eventos y la arquitectura en capas. La arquitectura basada en eventos es ideal cuando el sistema debe responder a estímulos en tiempo real, como sensores o señales externas, mientras que la arquitectura en capas conviene cuando se necesita separar claramente el control del hardware de la lógica de la aplicación.

Ejemplos de software embebido son los sistemas operativos en tiempo real (RTOS) como FreeRTOS, los sistemas de frenado antibloqueo en automóviles y las bombas de insulina (Finoit, 2025).

### **Observaciones y comentarios**

La elección de una arquitectura no depende únicamente de quién usará el software o cómo lo hará. Factores como el avance de la inteligencia artificial también influyen, ya que muchas aplicaciones hoy necesitan integrar modelos de IA, lo que exige arquitecturas capaces de escalar recursos bajo demanda, como microservicios o serverless.

La ciberseguridad es otro factor relevante: mientras más sensible sea la información que maneja una aplicación, más conviene una arquitectura que aisle componentes críticos, como el microker-

nel o los microservicios, frente a un sistema monolítico.

Esto muestra que, además de las necesidades funcionales de cada tipo de software, el contexto tecnológico actual también debe pesar en esta decisión.

## Conclusiones

1. El software se diseña para distintos tipos de usuarios, que pueden ser personas, otros sistemas o componentes electrónicos, y esto determina en gran parte qué arquitectura conviene usar.
2. Cada tipo de software (de sistema, de aplicación, de programación, middleware, controladores de dispositivo y embebido) responde a necesidades distintas, por lo que ninguna arquitectura funciona igual de bien para todos los casos.
3. Elegir la arquitectura adecuada no solo garantiza que el sistema funcione, sino que también facilita la escalabilidad y permite que el código sea más fácil de mantener y leer para el equipo de desarrollo.

## Bibliografía

- Dharmadasa, R. (2025, febrero). *Choosing the Right Software Architecture: A Guide to Selecting the Best Fit for Your Project*. Consultado el 5 de agosto de 2026, desde <https://ravindusandaruwandh.medium.com/choosing-the-right-software-architecture-a-guide-to-selecting-the-best-fit-for-your-project-ecdda7812fb1>
- Fazio, C. (2025, noviembre). *What are the Different Types of Software? Let's Break it Down*. Consultado el 5 de agosto de 2026, desde <https://distantjob.com/blog/types-of-software/>
- Finoit. (2025, febrero). *What is Software?* Consultado el 5 de agosto de 2026, desde <https://www.finoit.com/blog/software/>

URL del repositorio: [Repositorio del código Latex](#)